tailwindcss

Sass

ESGI 4IW 2025-2026

Edouard Sombié

Partie 1 : Sass

- Présentation
- Dart-sass (node.js)
- Sass vs scss
- Imbrication
- Variables
- Inclusion et import
- Boucles et conditions
- Fonctions
- Approche BEM
- Mise en pratique

D'autres pré-processeurs css

- less
- stylus
- postCSS

Présentation

Sass est un **pré-processeur css** permettant de compiler des feuilles de styles à partir de variables et de fonctions. Il permet également l'inclusion de fichiers (*partials*), rendant les projets d'intégration plus faciles à maintenir et à faire évoluer. Fini les "recherche et remplace" dans des fichiers css sans fin...
https://sass-lang.com/

La compilation peut être faite via des applications autonomes (ex. Codekit), mais aussi grâce à des bibliothèques JavaScript, Ruby, Swift ou Java.

Dans le cadre de ce cours, nous utiliserons la version js du compilateur qui tourne sur Node.js : le **package sass**, distribution de dart-sass.
https://www.npmjs.com/package/sass

Dart Sass (Node.js)

Installation globale (recommandée) :

```
npm install -g sass
```

```
Installation dans un projet :
npm install --save-dev sass
```

Exécution via une commande sass :

```
sass --watch [--style=compressed] input.scss output.css
```

Exécution via le fichier package.json (recommandée)

```
npm run watch
```

```
{
  "scripts": {
    "watch": "sass --watch input.scss output.css"
    }
}
```

Note :
sass, en tant qu'implémentation de dart-sass, lui ajoute une API js en plus du CLI natif.
https://sass-lang.com/documentation/js-api/

Sass vs scss

Les fichiers de compilation sass peuvent être écrits selon deux syntaxes différentes :

Sass "natif" (.sass)
Syntactically **A**wesome **S**tyle**S**heets

➔ Syntaxe basée sur l'indentation.

```
.button
  display: block
  padding: 1em
```

Scss (.scss)
Sassy **C**ascading **S**tyle**S**heets

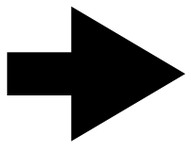
➔ Syntaxe identique au css
(accolades, point-virgule)

```
.button{
  display: block;
  padding: 1em;
}
```

Bien entendu, nous utiliserons la **syntaxe scss** qui nous permettra d'inclure des bibliothèques déjà compilées ou des composants déjà écrits en css ! 😎

Imbrication (nesting)

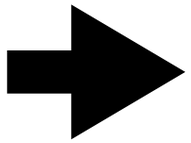
```
body{
    margin: 0;
    h1{
        text-transform: uppercase;
    }
}
```



```
body{
    margin: 0;
}
body h1{
    text-transform: uppercase;
}
```

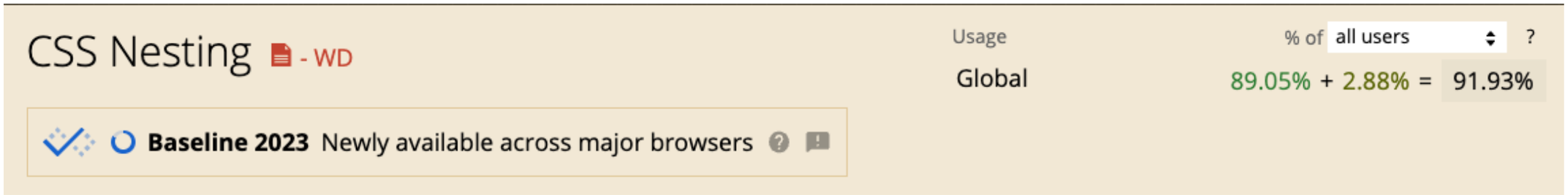
& "le sélecteur de parent"

```
ul{
    display: flex;
    aside &{
        flex-direction: column;
    }
}
```



```
ul{
    display: flex;
}
aside ul{
    flex-direction: column;
}
```

A noter :
L'imbrication est possible aujourd'hui en css natif.
A éviter pour le moment pour garantir un bon support navigateur complet.



Source: https://caniuse.com/css-nesting

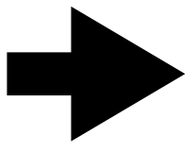
```
body{
    margin: 0;
    h1{
        text-transform: uppercase;
    }
    &:hover{
        background-color: black;
    }
}
```

Mais aucun intégrateur digne de ce nom n'écrit du css non compilé... 🤓🤔

Variables

```
$base-color: #c6538c;
$border-dark: rgba($base-color, 0.88);
$space: 10px;

.alert {
  border: 1px solid $border-dark;
  padding : 3px + $space;
}
```



```
.alert {
  border: 1px solid rgba(#c6538c, 0.88);
  padding : 13px;
}
```

On assigne une valeur par défaut à une variable avec le drapeau **!default**

```
$main-color : black !default;
```

Note :
Dans les noms de variables, - et _ sont traités de manière identique.
$ma_variable == $ma-variable 🤔

Portée des variables
Les variables déclarées au premier niveau de la feuille de style sont globales.
Celles déclarées dans un sélecteur (entre accolades) sont locales.

Une variable globale existante est modifiée depuis un scope local avec **!global**

```
$main-color : red !global;
```

Types de variables	Modules associés

Number
Deux composants : le nombre lui-même et l'unité
`5.2e3, 16px`
`$value-with-unit : $number * 1px`
	sass:math
	`math.div(100px, 2s)`
	`math.ceil(4.9);`

String
Avec ou sans guillemets
`"Roboto", bold`
	sass:string
	`string.length("Helvetica")`
	`string.unique-id()`

Color
Hexa, rgba, hsla, mot-clé
`red, #777`
	sass:color
	`color.blackness(#e1d7d2)`
	`color.scale(#036, $lightness: -30%)`

List
Tableau de valeurs
`Helvetica, Arial, sans-serif`
`10px 15px 0 0`
`[line1 line2]`
	sass:list
	`list.append(10px 20px, 30px)`
	`list.nth(10px 12px 16px, 2)`

Map
Tableau de clés : valeurs
`("reg": 400, "med": 500, "bold": 700)`
	sass:map
	`map.deep-remove($weights, "reg")`
	`map.get($weights, "medium")`

Calculation Fonctions css de calcul `calc(), min(), max(), clamp()`	

Function
Fonction avec valeur de retour
`@function rem($val){`
`  @return calc($val / 16 * 1rem);`
`}`	

Boolean	
Null	

Concaténation de variables

Pour concaténer la valeur d'une variable (interpolation), on utilise **#{ }**
Cela est utile lorsque l'on veut concaténer une valeur dans une chaîne de caractères.

```
--bg-color : #{$secondary-color};
.col-#{$i} {...}
```

Attention !
Lors de la concaténation, les guillemets d'une valeur string sont supprimés.
Pour éviter cela, il faut envelopper la valeur dans une fonction **meta.inspect()**

```
@use "sass:meta";

$font-family-monospace: Menlo, Consolas, "Courier New", monospace;

:root {
  --font-family-monospace: #{meta.inspect($font-family-monospace)};
}
```

Note : il a fallu ici importer le module sass:meta avec @use. Nous allons voir cela bientôt...

Opérations sur des variables

Deux variables ayant la même unité peuvent être additionnées ou soustraites l'une à l'autre.

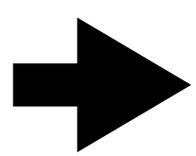
```
$padding-base: 1rem;

.btn {
  padding : $padding-base + 0.5rem;
}
```

Les variables peuvent également être multipliées par une valeur.
Par contre, pour une division, il faut utiliser **calc()** ou **math.div()** ;

```
@use "sass:math";
$space: 2rem;

.button{
    margin: $space * 2;
    padding: math.div($space, 2);
    line-height: calc($space / 2);
    font-size: calc(#{$space} /
2);
}
```



```
.button {
  margin: 4rem;
  padding: 1rem;
  line-height: 1rem;
  font-size: calc(2rem / 2);
}
```

Noms de variables dynamiques

Pour faire référence à un nom de variable en fonction d'une autre variable, on peut, comme dans d'autres langages, passer par un tableau de variables.

```
@use "sass:map";

$theme-colors: (
  "success": #28a745,
  "info": #17a2b8,
  "warning": #ffc107,
);

.warning {
  background-color: map.get($theme-colors, "warning");
}
```

Variables sass vs variables css

Les variables sass sont très différentes des **variables css**, dans la mesure où elles ne sont pas accessibles en *front end,* elles disparaissent lors de la compilation.
Elles sont utilisées dans le code lorsqu'il est nécessaire de leur appliquer des opérations (color.scale, map.get, fonctions custom...).

Pour que certaines valeurs soient accessibles du côté *front end*, on injecte des variables sass dans des variables css, ce qui facilite la création de composants variables (dark/light par exemple).
Se reposer sur des valeurs contenues dans des variables css, appelées aussi "propriétés personnalisées", est une **très bonne pratique**.

```
@use "sass:color";
$pink : #ff2896;

:root{
  --pink: #{$pink};
  --pink-light: color.scale( $pink, $lightness: 30%);
}

a{
  color: var(--pink);
  &:hover{
    color: var(--pink-light);
  }
}
```

Les variables css permettent notamment de changer le comportement d'un élément en fonction de son **contexte**.

```
@use "sass:color";
$pink : #ff2896;
$blue: #1e293a;

:root{
  --linkColor: #{$pink};
  --linkColor-light: color.scale( $pink, $lightness: 30%);
}

.footer{
  --linkColor: #{$blue};
  --linkColor-light: color.scale( $blue, $lightness: 30%);
}

a{
  color: var(--linkColor);
  &:hover{
    color: var(--linkColor-light);
  }
}
```

Import et inclusion

@import
Cette règle permet d'importer des fichiers scss externes, des *partials* (préfixés d'un underscore) ou des feuilles de styles css.

```
@import "variables"; // fait référence au fichier _variables.scss
@import "vendor/bootstrap-min";
```

Notes :

- Pas besoin de spécifier ./ pour les chemins relatifs.
- La règle **@import est en principe destinée au premier niveau** d'une feuille de style mais peut également être appelée dans un sélecteur.
  ```
  .btn { @import "btn-rules"}
  ```

Attention ! 🖐
Bien que très utile dans le cadre d'un framework simple articulé autour d'un fichier *master*, **l'utilisation de cette règle est aujourd'hui déconseillée** à cause de certains problèmes qu'elle peut engendrer :

- Un *partial* est importé à chaque appel de la règle @import
- Les variables, fonctions et *mixins* qu'il contient sont globaux, ce qui peut être source de conflit et oblige souvent à préfixer les sélecteurs d'une bibliothèque.

La solution ? **@use** 😎

@use

Comme @import, cette règle permet d'importer des fichiers scss externes, des *partials* (préfixés d'un underscore) ou des feuilles de styles css. Les sources incluses par @use sont appelées **modules**.

```
@use 'foundation/code'; _code.scss
@use "vendor/bootstrap-min";
```

Notes :

- La règle @use doit être appelée **avant toute règle de style**.
- A la grande différence de @import, les variables, fonctions et *mixins* d'un fichier chargé par @use appartiennent à un ***namespace*** (identique au nom du fichier).

```
// src/_borders.scss
$radius: 3px;


// _card.scss
@use "src/borders";
.card{
    border-radius : borders.
$radius;
}
```

Namespace *custom*

```
// _card.scss
@use "src/borders" as b;
.card{
    border-radius : b.$radius;
}

// _card.scss
@use "src/borders" as *;
.card{
    border-radius : $radius;
}
```

Modules sass natifs (cf page sur les types de variables)

```
sass:color, sass:list, sass:map, sass:math, sass:meta,
sass:selector, sass:string
```

```
@use "sass:map";
@media screen and (min-width: #{map.get($breakpoints, lg)}) {...}
```

Configuration de module

Lors de l'importation d'un module, il est possible d'en modifier la configuration, c'est à dire redéfinir les variables et sélecteurs qu'il contient. On utilise l'instruction **with**.

```
// _library.scss
$black: #000 !default;
$padding: 1rem !default;

@use "library" with (
  $black: #222,
  $padding: 2rem
)
```

Variables privées

Pour limiter la visibilité d'une variable à un module, il faut la préfixer avec - ou _

```
$-radius: 3px;
```

@forward

Cette instruction très utile permet de charger les variables, fonctions et mixins d'une feuille de style A vers une feuille de style B qui seront disponibles à l'usage de cette mêle feuille de style B

```
// styleA.scss
@mixin list-reset {
  margin: 0;
  padding: 0;
  list-style: none;
}

// styleB.scss
@forward "styleA";

// styles.scss
@use "styleB";

li {
  @include styleB.styleA-list-reset;
}
```

Avec un préfixe custom :

```
// stylesheetA.scss
@mixin list-reset {
  margin: 0;
  padding: 0;
  list-style: none;
}

// stylesheetB.scss
@forward "stylesheetA" as s-*;

// styles.scss
@use "stylesheetB";

li {
  @include stylesheetB.s-list-reset;
}
```

Lors d'un @forward, on peut masquer des membres :

```
@forward "src/list" hide list-reset, $horizontal-list-gap;
```

Réutilisation de code (*snippets*)

@mixin

Les mixins sont une des fonctionnalités les plus puissantes de sass. Ils permettent l'inclusion de blocs de code css grâce à l'instruction **@include**.

```
@mixin resetList {
    list-style: none;
    margin: 0;
    padding: 0;
}

ul {
    @include resetList;
}
```

```
@mixin resetList($padding) {
    list-style: none;
    margin: 0;
    padding: $padding;
}

ul {
    @include resetList(1rem);
}
```

Les mixins peuvent recevoir des **paramètres**.

@extend 😨

Cette instruction, assez proche de @include permet d'étendre une classe ou un placeholder. **Son utilisation est fortement déconseillée**, au profit des mixins, car elle perturbe la structure de la feuille de style et crée des liens entre sélecteurs qui peuvent générer de effets indésirables.

```
.no-border {
    border: none;
}

.maClasse {
    font-size: 1rem;
    @extend .no-border;
}
```

```
.no-border, .maClasse {
  border: none;
}

.maClasse {
  font-size: 1rem;
}
```

Avec un **placeholder %**, le sélecteur de base n'est pas compilé.
Mais ça reste pas terrible du tout !

```
%no-border {
    border: none;
}

.maClasse {
    font-size: 1rem;
    @extend %no-border;
}
```

```
.maClasse {
  border: none;
}

.maClasse {
  font-size: 1rem;
}
```

Boucles et conditions

Sass offre les traditionnelles boucles **@for** et **@while**.

- ```
  @for $i from 1 through 6 {
      .p-#{$i} {
          padding : $i * 0.5rem;
      }
  }
  ```

  // la dernière itération est incluse

- ```
  @for $i from 1 to 7 {
      .p-#{$i} {
          padding : $i * 0.5rem;
      }
  }
  ```

  // la dernière itération est exclue

- ```
  @while $value > $base {
    $value: math.div($value, $ratio);
  }
  @return $value;
  ```

  // **Cette boucle est à éviter** si possible, au profit de @for ou @each car elle augmente le temps de compilation et repose sur des valeurs true/false assez strictes (0 != false par exemple)
  ```

La boucle **@each** pour parcourir listes et maps

- ```
  @each $size in $sizes {
      .icon-#{size} {
          font-size: $size;
      }
  }
  ```
  // liste

- ```
  @each $name, $size in $breakpoints {
      .container-#{name} {
          width: $size;
      }
  }
  ```
  // map

Le cas d'une liste de listes (**destructuration**)

```
$icons:
  "eye" "\f112" 12px,
  "start" "\f12e" 16px,
  "stop" "\f12f" 10px;

@each $name, $glyph, $size in $icons {
  .icon-#{$name}:before {
    content: $glyph;
    font-size: $size;
  }
}
```

Les conditions

Elles peuvent être évaluées avec les règles suivantes :

- `@if{}`
- `@else{}`
- `@else if{}`

Que du classique...

```
@if $light-theme {
    background-color: $light-background;
    color: $light-text;
  } @else {
    background-color: $dark-background;
    color: $dark-text;
  }
```

Fonctions

Sass propose de nombreuses fonctions natives permettant de modifier des valeurs ou d'évaluer des variables.

Certaines sont globales (en voie de dépréciation) :

```
darken($red, 30%);
```

D'autres font partie de modules sass natifs :

```
color.mix($color1, $color2, $weight: 50%);
```

Liste complète : https://gist.github.com/AllThingsSmitty/3bcc79da563df756be46

@function

On peut également créer des **fonctions *custom*** qui traiteront une valeur d'entrée afin de renvoyer un résultat.

```
@use "sass:math";
@function rem($pxVal) {
    @return math.div($pxVal, 16) * 1rem;
}
```

L'approche BEM

Même si elle ne date pas d'hier, cette méthode est considérée comme une bonne pratique (pas que par moi...), elle repose sur les notions de :

- **Block**
  entité autonome
  `.card` ou `.navigation`

- **Element**
  partie d'un block, y faisant référence directement
  `.card__title` ou `.navigation__item`

- **Modifier (modificateur)**
  "drapeau" appliqué à un block ou un élément permettant de modifier son apparence ou on comportement
  `.card--dark` ou `.navigation--vertical`

Elle permet de créer un cadre rigide de nommage et de s'affranchir des problèmes causés par les styles en cascade (génère une forme de scope local des règles).

Element

```
.card{
    display: flex;
    flex-direction: column;
    &__title{
        font-size: 1.2rem;
    }
    &__image{
        object-fit: cover;
    }
}
```

```
.card{
    display: flex;
    flex-direction: column;
}
.card__title{
    font-size: 1.2rem;
}
.card__image{
    object-fit: cover;
}
```

Modificateur

```
.btn{
    font-size: 1rem;
    padding: 1em;
    &--sm{
        font-size: 0.7rem;
    }
    &--lg{
        font-size: 2rem;
    }
}
```

```
.btn{
    font-size: 1rem;
    padding: 1em;
}
.btn--sm{
    font-size: 0.7rem;
}
.btn--lg{
    font-size: 2rem;
}
```

Le cas du "petit-enfant"

L'approche BEM permet une écriture simple des composants et une lecture claire du code css généré. Toutefois, l'imbrication de classes peut amener à des noms à rallonge du type `.navigation__item__link__icon`

Voici ce qu'en dit le créateur de la méthode :

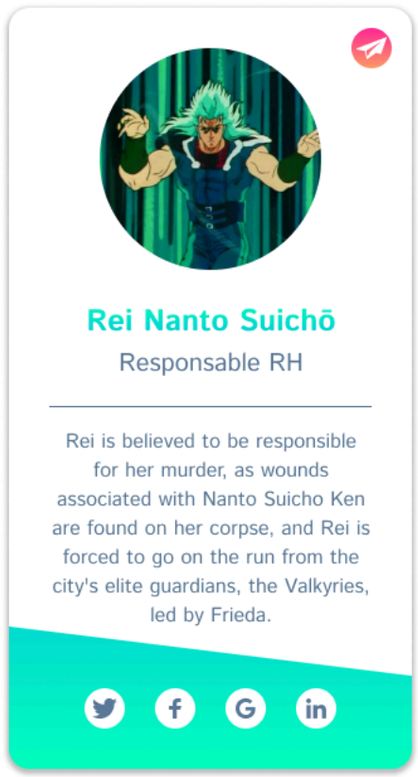
> “BEM methodology doesn’t recommend to use elements within elements in class names. You don’t need to resemble DOM structure in naming. Having one level structure makes refactoring much easier.”
> — Vladimir Grinenko, Yandex

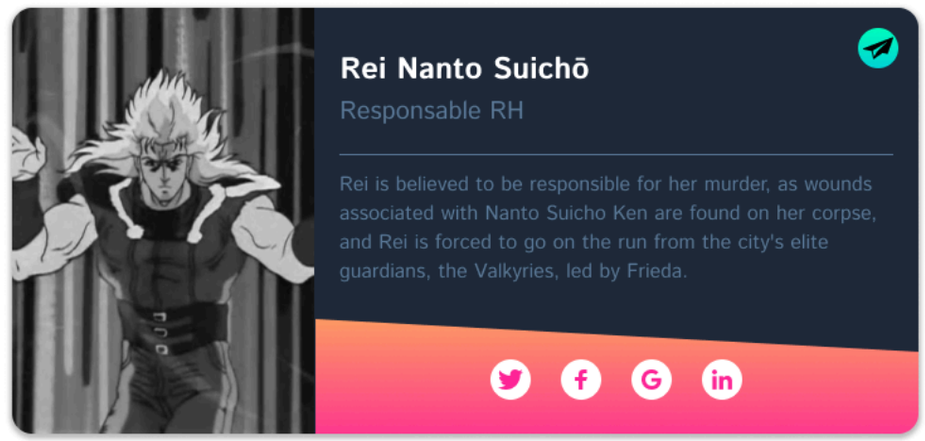
```
.navigation{
    /* */
    &__item{
        /* */
        &__link{
            /* */
            &__icon{
                /* */
            }
        }
    }
}
```

```
.navigation{
    /* */
    &__item{
        /* */
    }
    &__link{
        /* */
    }
    &__icon{
        /* */
    }
}
```

Mise en pratique

C'est le moment d'appliquer ce que l'on vient de voir :
https://www.figma.com/file/HV3Sc7JoEMGcR0gCkvUcU3/TP1-sass?type=design&node-id=0-1&mode=design&t=KLZHxAeuaRUg8npC-0





Pour les consignes et le chemin à suivre, rendez-vous sur myGES, dans la rubrique supports de cours : TP1-sass 🤟

D'autres pré-processeurs

Bien entendu, sass n'est pas le seul pré-processeur css utilisé aujourd'hui, bien qu'étant le plus populaire. Voici trois de ses "concurrents"... histoire de soigner sa culture générale.

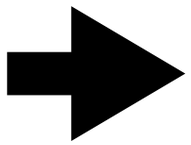
https://lesscss.org/

Leaner CSS
Mixins, variables, fonctions, bibliothèques, très proche de sass mais moins utilisé...

```
> lessc styles.less styles.css
```

```
@bg: black;
@bg-light: boolean(luma(@bg) > 50%);

div {
  background: @bg;
  color: if(@bg-light, black, white);
}
```



```
div {
  background: black;
  color: white;
}
```

D'autres pré-processeurs

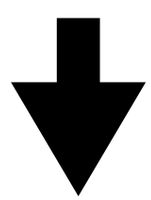
https://stylus-lang.com/

Syntaxe très (trop ?) souple, gestion des prefixes, commandes CLI très complètes, plugins...

```
> stylus --compress < some.styl > some.css
```

```
font-size = 14px
font-stack = "Lucida Grande", Arial, sans-serif

body
  font font-size font-stack
```

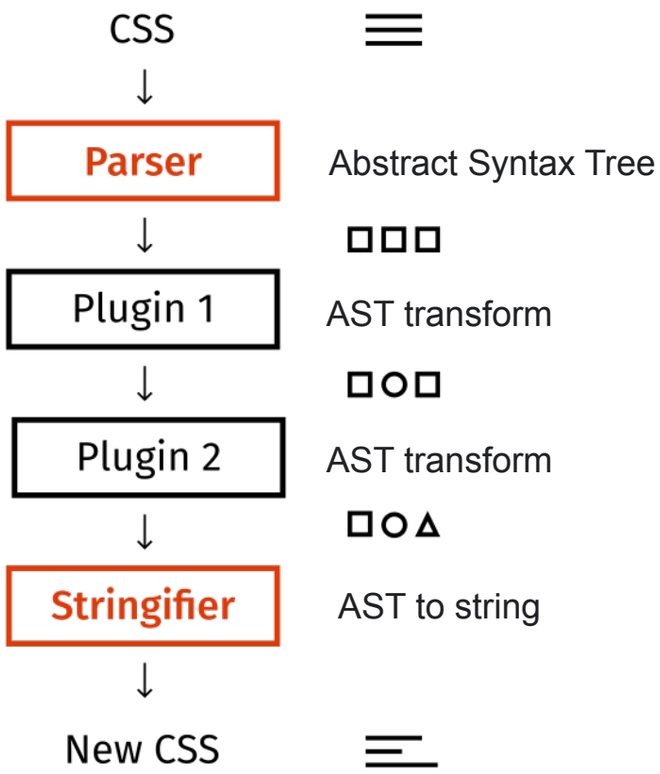


```
body {
  font: 14px "Lucida Grande", Arial, sans-serif;
}
```

D'autres pré-processeurs

https://postcss.org/

Pas réellement un pré-processeur comme sass ou less, plutôt un **outil js de manipulation de la syntaxe css**.
Input css, autoprefixer, modules css, plugins...
Le pré-processeur sur lequel repose **Tailwindcss 4** 👌




```
> postcss --use autoprefixer -o main.css css/*.css
```

```
.cfx {
  @util clearfix;
}

.rounded-top {
  @util border-top-radius(4px);
}
```

```
.cfx::after {
  content: '';
  display: block;
  clear: both;
}

.rounded-top {
  border-top-left-radius: 4px;
  border-top-right-radius: 4px;
}
```

D'autres pré-processeurs

https://lightningcss.dev/

Tout comme postCSS, il s'agit d'**outil js de manipulation de la syntaxe css**.
Transpilation, minification, bundling...
Le transpileur sur lequel repose **Tailwindcss 4** 👌

```
> lightningcss --minify --bundle --targets \">= 0.25%\" input.css -o output.css
```

Maintenant que l'on a vu un outil qui permet de créer des frameworks css, remontons d'un niveau et intéressons-nous à un framework déjà compilé...

tailwindcss

Tailwind css

- Présentation
- Créer un projet Tailwind css
- Les classes "utilitaires"
- Customisation de classe
- Les modificateurs de classe
- Customisation des modificateurs
- Mise en pratique

  ...

tailwindcss

Présentation

Framework css "bas-niveau" basé sur des **utility classes**, créé en 2017 par Adam Wathan. La version actuelle est v**4.0**

`text-sm, rounded-lg, pt-2`

Considéré comme "bas-niveau", ce framework ne propose aucun composant prêts à l'emploi, hormis un container qui n'est pas centré 🤔...
Pour l'utiliser, il faut connaître les règles CSS. Depuis la version 4, il n'est plus nécessaire d'utiliser de fichiers js (config ou plugins) pour customiser le framework, tout se fait au niveau du fichier input.css.

Comme vu précédemment, ce framework est basé sur **postCSS** et **lightningCSS**. La compilation se fait via node, grâce à des outils tels que vite.js, postCSS ou tailwind CLI.

Il est clairement le framework css le plus en vogue actuellement de par sa vitesse de mise en œuvre. Il est certainement une très bonne solution pour la création rapide d'une interface et apparaît (à mon humble avis) comme un outil complémentaire de sass qui, pour sa part, offre des fonctionnalités plus puissantes mais certains écueils aussi (beaucoup à écrire, nommage...).

https://tailwindcss.com/

SASS

Les +

- Markup html plutôt léger et lisible
- Fonctions, mixins, conditions et boucles 💪
- Imbrication de règles, génération de sélecteurs
- On peut tout faire ! Une approche BEM ou des utility classes, ou les deux !

Les -

- Tout le css doit être écrit *from scratch*
- Règles de nommage vite confuses : composant agnostique ou pas ? utilitaire ou pas ? 🤔
- On peut tout faire, donc aussi du "pas fou fou"
- Markup et style se font séparément
- Comme avec du css traditionnel, on peut se retrouver à batailler contre des règles globales (avant les @layers css)

Tailwind css

Les +

- Utility classes propices à une intégration cohérente (spacing par 0.25rem)
- CSS plus compact : pas toujours nécessaire d'ajouter du code *custom*
- Plus de css mort grâce à *JIT*
- Pas de souci de contexte : le style est dans le html
- Pas de souci de nommage : on se repose uniquement sur des utility classes
- Pas d'effet de bord lors de changements sur un composant
- L'intégration peut être beaucoup plus rapide
- Plus de styles en cascades... parfois source de bugs ou d'ajout de *!important* inopportuns
- Se combine très bien aux composants des frameworks front tels que React, Vue ou Svelte 🤩

Les -

- Markup html vite **très touffu**
- Utility classes peuvent engendrer un manque de cohérence entre les composants...
- Nécessaire de se familiariser avec les utility classes avant d'être productif
- Plus de styles en cascades... parfois pratiques 😅

Vous avez dit "touffu" le html ?

Save

```
<button type="submit" class="rounded-md bg-indigo-600 px-3
py-2 text-sm font-semibold text-white shadow-sm hover:bg-
indigo-500 focus-visible:outline focus-visible:outline-2
focus-visible:outline-offset-2 focus-visible:outline-
indigo-600">Save</button>
```

Tailwind css

```
<button type="submit" class="btn btn--indigo">Save</button>
```

BEM :)

Il ne va pas falloir avoir peur de faire des copier/coller. 🥵
Et j'espère qu'on ne va pas me demander de modifier le border-radius des boutons de mon site... 😭

Tailwind css propose bien une directive **@apply**, mais pour le coup, l'utiliser est vraiment un non-sens... On en reparlera.

Surtout **Tailwind css est fait pour être utilisé avec des framework *front-end* basés sur des composants autonomes** ! React, Vue, Svelte ou des moteurs comme Blade...

Mais un css optimisé !

Grâce à une compilation *Just in Time* (**JIT**), **seules les classes utilisées sont écrites dans le fichier compilé.** En effet, lors de la compilation, Tailwind css passe en revue toutes les classes contenues dans les templates du projet et ne conserve que celles-ci !

Tailwind offre également la possibilité d'importer une base de feuille de styles reposant sur *modern-normalize* appelée ***preflight***.
https://tailwindcss.com/docs/preflight

```
@layer theme, base, components, utilities;

@import "tailwindcss/theme.css" layer(theme);
@import "tailwindcss/preflight.css" layer(base);
@import "tailwindcss/utilities.css" layer(utilities);
```

Note :
Dans la v3 du framework, il était possible de forcer la compilation de classes grâce au paramètre **safelist.**
Ce n'est plus le cas dans la v4, pour obtenir le même résultat, il faudra que les classes à compiler soient présentes dans un template (même en commentaire).

Créer un projet Tailwind css

Installation (via node.js) pour un projet vite

https://tailwindcss.com/docs/installation/using-vite

https://vite.dev/guide/

```
> npm create vite@latest
> npm install
> npm install tailwindcss @tailwindcss/vite
```

Déclaration du plugin tailwind (dans le fichier vite.config.js)

```
import { defineConfig } from 'vite'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [
    tailwindcss(),
  ],
})
```

Ajout des *directives* au fichier source (/src/style.css)

```
@import "tailwindcss";
```

Créer un projet Tailwind css

Définir dans la config de vite.js les chemins de compilation (css/main.css)

```
import { defineConfig } from "vite";
import tailwindcss from "@tailwindcss/vite";
export default defineConfig({
    plugins: [tailwindcss()],
    build: {
        minify: false,
        rollupOptions: {
            output: {
                assetFileNames: (assetInfo) => {
                    if (/\.css$/.test(assetInfo.names[0])) {
                        return "css/[name][extname]";
                    }
                    return "assets/[name][extname]";
                },
            },
        },
    },
});
```

Ecouter les modifications du fichier source

```
> npm run dev
```

Note : cette instruction lance également un serveur local qui permet de consulter le résultat de la compilation.

Créer un projet Tailwind css

Il existe bien d'autres manières d'utiliser tailwindcss.

Notamment dans le cadre de frameworks front.

https://tailwindcss.com/docs/installation/framework-guides

Next.js

Full-featured React framework with great developer experience.

Laravel

PHP web application framework with expressive, elegant syntax.

Nuxt

Intuitive Vue framework for building universal applications.

SolidJS

A tool for building simple, performant, and reactive user interfaces.

SvelteKit

The fastest way to build apps of all sizes with Svelte.js.

Angular

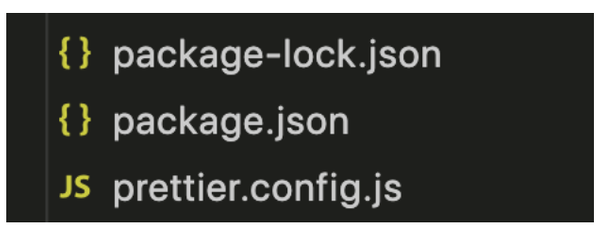
Platform for building mobile and desktop web applications.

Des outils utiles, voire indispensables

On va commencer par utiliser Visual Studio Code...
C'est là que l'on retrouve les extensions (officielles) les plus utiles :

- **Tailwind CSS IntelliSense**
  autocomplete, linting, hover preview
  https://marketplace.visualstudio.com/items?itemName=bradlc.vscode-tailwindcss

- **Prettier + prettier-plugin-tailwindcss**
  Auto-format html/css + tri des classes tailwind*
  https://marketplace.visualstudio.com/items?itemName=esbenp.prettier-vscode
  https://github.com/tailwindlabs/prettier-plugin-tailwindcss



*Etant donné le nombre importants de classes Tailwind css sur un élément, il faut suivre l'ordre de classes préconisé par tailwind css. https://tailwindcss.com/blog/automatic-class-sorting-with-prettier#how-classes-are-sorted

Sans oublier la liste des classes : **https://tailwindcomponents.com/cheatsheet/**
Ces outils font référence à la v3 mais font le job...

Principes de base

- Layers css natifs
  theme, base, components, utilities

- Directives
  https://tailwindcss.com/docs/functions-and-directives#directives

- Classes utilitaires
  https://tailwindcss.com/docs/styling-with-utility-classes

- Variables de thème
  https://tailwindcss.com/docs/theme

- Variants
  https://tailwindcss.com/docs/hover-focus-and-other-states

- Mobile first
  https://tailwindcss.com/docs/responsive-design

- Dark mode compatible
  https://tailwindcss.com/docs/dark-mode

Les directives

Depuis la version 4 du framework, plus besoin d'un fichier de config js (toujours supporté par souci de compatibilité).

La configuration du framework se fait désormais **via le css**.
Pour cela, on utilisera dans l'input css des ***directives*** :

@import

@theme

@source

@utility

@variant

@custom-variant

@apply

@reference

Les directives

La customisation repose sur du css natif mais est structurée par des **@directive**

app.css

```
@import "tailwindcss";

@theme {
  --color-mint-500: oklch(0.72 0.11 178);
}
```

Par exemple, la détection de classes se fait sur tous les fichiers du projet, mais il est possible d'ajouter un chemin *custom*.

CSS

```
@import "tailwindcss";
@source "../node_modules/@acmecorp/ui-lib";
```

Les classes utilitaires

Tailwind css propose des centaines de ***utility classes***.
L'objet du cours n'est pas de les lister ici...

Elles sont toutes décrites ici : https://tailwindcss.com/docs/

On retiendra qu'elles sont classées par type :

- Layout
- Flexbox & grid
- Spacing
- Sizing
- Typography
- Backgrounds
- Borders
- Effects
- Filters
- Tables
- Transitions & animation
- Transforms
- Interactivity
- SVG
- Accessibility

Les variables de thème

Les classes utilitaires reposent sur des variables css (custom properties) contenues dans la directive @theme
Il suffit d'ajouter de nouvelles variables pour étendre les classes qui s'y rattachent.

https://tailwindcss.com/docs/adding-custom-styles#customizing-your-theme

```css
@theme {
  --font-display: "Satoshi", "sans-serif";

  --breakpoint-3xl: 120rem;

  --color-avocado-100: oklch(0.99 0 0);
  --color-avocado-200: oklch(0.98 0.04 113.22);
  --color-avocado-300: oklch(0.94 0.11 115.03);
  --color-avocado-400: oklch(0.92 0.19 114.08);
  --color-avocado-500: oklch(0.84 0.18 117.33);
  --color-avocado-600: oklch(0.53 0.12 118.34);

  --ease-fluid: cubic-bezier(0.3, 0, 0, 1);
  --ease-snappy: cubic-bezier(0.2, 0, 0, 1);

  /* ... */
}
```

Les variables de thème

Pour ajouter une valeur custom à des classes utilitaires, il faut déclarer cette valeur dans le *namespace* de ces classes.

Par exemple, pour ajouter une nouvelle valeur de couleur, il faut le faire dans le namespace `--color`
Dès lors, toutes les classes qui utilisent ce namespace existeront avec cette valeur.

```
@theme {
    --color-esgi: red;
}

<div class="bg-esgi"></div>
```

Note : il est possible de supprimer des namespaces

```
@theme {
     --*: initial; // supprimer toutes les valeurs
     --color-red-*: initial; // supprimer toutes les valeurs de rouge
}
```

Les variants de classe

Tailwind css propose des modificateurs de classe du type **var:class**
Ils permettent d'appliquer des styles sur des :

- pseudo-classes
  `hover: focus: first: even:`

- pseudo-éléments
  `before: after:`

- media queries
  `md: xl:`

- sélecteurs d'attribut
  `aria-disabled: data-[size=large]:`

Note :
Les modificateurs peuvent être imbriqués.
`md:aria-disabled:dark:`

Customisation des variants

En plus des modificateurs de base de Tailwind, il est possible d'ajouter ceux de notre choix, soit directement dans le markup html soit grâce à une déclaration dans le fichier d'input css.

- dans le markup html
  ```
  <li class="[&:nth-child(3)]:underline">{item}</li>
  ```

- dans le fichier css
  ```
  @custom-variant theme-midnight (&:where([data-theme="midnight"] *));
  ```

Le *responsive design*

Tailwind css propose des sélecteur permettant une approche ***mobile first***.

Breakpoint prefix	Minimum width	CSS
`sm`	40rem *(640px)*	`@media (width >= 40rem) { ... }`
`md`	48rem *(768px)*	`@media (width >= 48rem) { ... }`
`lg`	64rem *(1024px)*	`@media (width >= 64rem) { ... }`
`xl`	80rem *(1280px)*	`@media (width >= 80rem) { ... }`
`2xl`	96rem *(1536px)*	`@media (width >= 96rem) { ... }`

Les styles 'responsive' sont appliqués à toutes les classes via des ***variants*** type **:md** ou **:lg**

```
<div class="md:flex">
```

Note : Il est aussi possible de définir une étendue (range) d'application des styles.

```
<div class="md:max-lg:flex">
```

Le *dark mode*

Tailwind prend en charge les styles associés à un mode sombre du navigateur.
Par défaut, cette prise en charge repose sur la media query **prefers-color-scheme**.
Pour appliquer un style 'dark', il faut utiliser le modificateur **dark:**

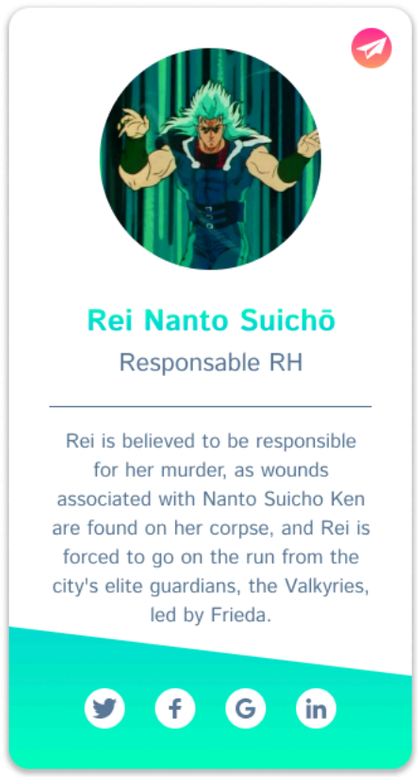
```
<h3 class="... dark:text-white ...">Writes Upside-Down</h3>
```

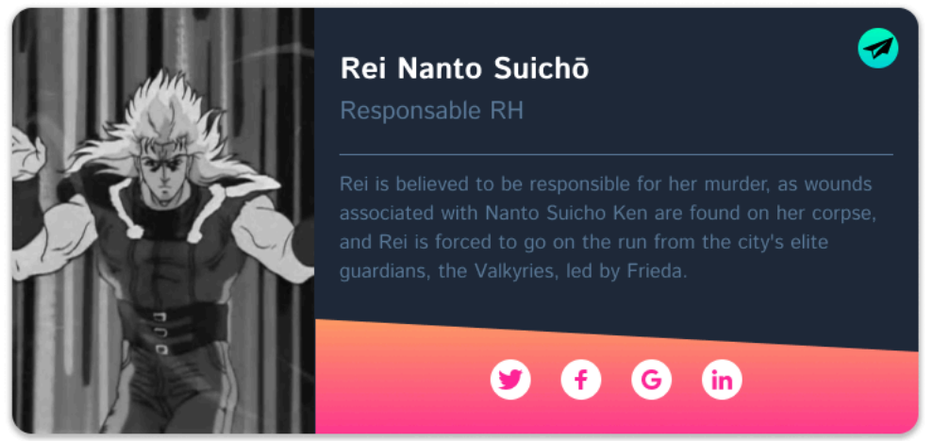
Il est toutefois possible d'associer l'application de styles 'dark' à une classe ou un attribut **dark**, il faut pour cela faire une déclaration dans le thème Tailwind

```
@custom-variant dark (&:where(.dark, .dark *));
@custom-variant dark (&:where([data-theme=dark], [data-theme=dark] *));
```

Mise en pratique

C'est le moment d'utiliser le framework pour recréer notre composant du TP1 !
https://www.figma.com/file/HV3Sc7JoEMGcR0gCkvUcU3/TP1-sass?type=design&node-id=0-1&mode=design&t=KLZHxAeuaRUg8npC-0





Pour les consignes et le chemin à suivre, rendez-vous sur myGES, dans la rubrique supports de cours : TP2-tailwind 🤘

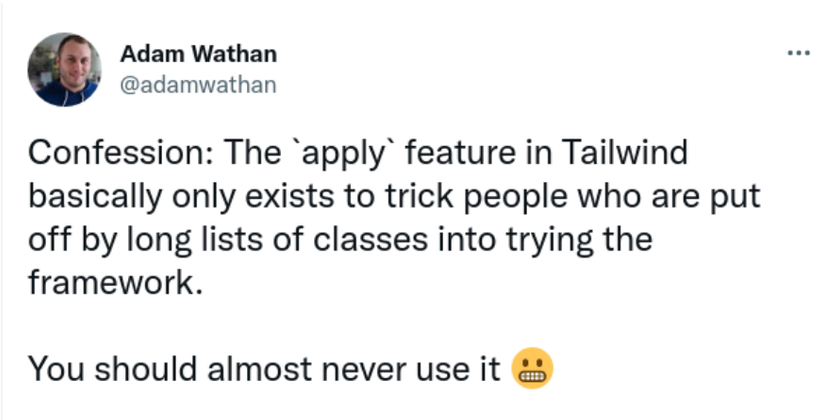
La directive @apply

Cette directive permet de créer des abstractions de classe.

```
@layer components {
  .btn-primary {
    @apply py-2 px-4 bg-blue-500 text-white font-semibold...;
  }
}
```

Note : @apply doit être utilisé avec parcimonie (pour de tous petits composants) car il va à l'encontre du principe de Tailwind et revient à créer des composants css...

La directive @apply



Confession : la fonctionnalité « apply » dans Tailwind n'existe essentiellement que pour inciter les personnes découragées par de longues listes de classes à essayer le framework.

Vous ne devriez presque jamais l'utiliser 😬

Bien que son usage soit déconseillé, cette directive est à la base de certains frameworks "orientés composants" comme DaisyUI.

`carousel`	Component	Container element
`carousel-item`	Component	Carousel item
`carousel-center`	Modifier	Snap elements to center
`carousel-end`	Modifier	Snap elements to end
`carousel-vertical`	Modifier	Vertical carousel

Customisation de classe dans le markup html

En plus de la modification des classes du thème, il est possible d'appliquer des styles "arbitraire" que ce soit au niveau de la valeur, de la propriété ou du sélecteur.

- valeur arbitraire
  `top-[117px] bg-[#bada55] bg-(--my-color)`

- propriété arbitraire (y compris variable css)
  `[mask-type:luminance] [--scroll-offset:56px]`

- modificateur arbitraire
  `[&:nth-child(3)]:hover:underline`

Il est parfois nécessaire d'utiliser des espaces dans une classe ou propriété custom. Dans ce cas, un utilise un underscore (_)
`grid-cols-[1fr_500px_2fr]`

On va éviter le plus possible d'utiliser ça !!! 🫨

Appliquer du style en fonction des parents

La classe .group

Il arrive souvent d'avoir à modifier l'aspect d'un élément en fonction de l'état de son parent. Ainsi, on voudra qu'un sous-menu apparaisse au survol de la <li> qui le contient. Pour cela, tailwind nous propose la classe **.group**

```
<li class="group">
    <ul class="hidden group-hover:flex"></ul>
</li>
```

Il est possible de nommer les groupes :

```
<li class="group/principal">
    <ul class="hidden group-hover/principal:flex"></ul>
</li>
```

Comme on le voit dans cet exemple, la classe .group permet d'utiliser des variants du type **group-XXX:**class

Appliquer du style en fonction des frères (*siblings*)

La classe .peer

Comme précédemment, on peut modifier l'apparence d'un élément en fonction de l'état des éléments frères qui le précèdent. Pour cela, on utilisera la classe **.peer**

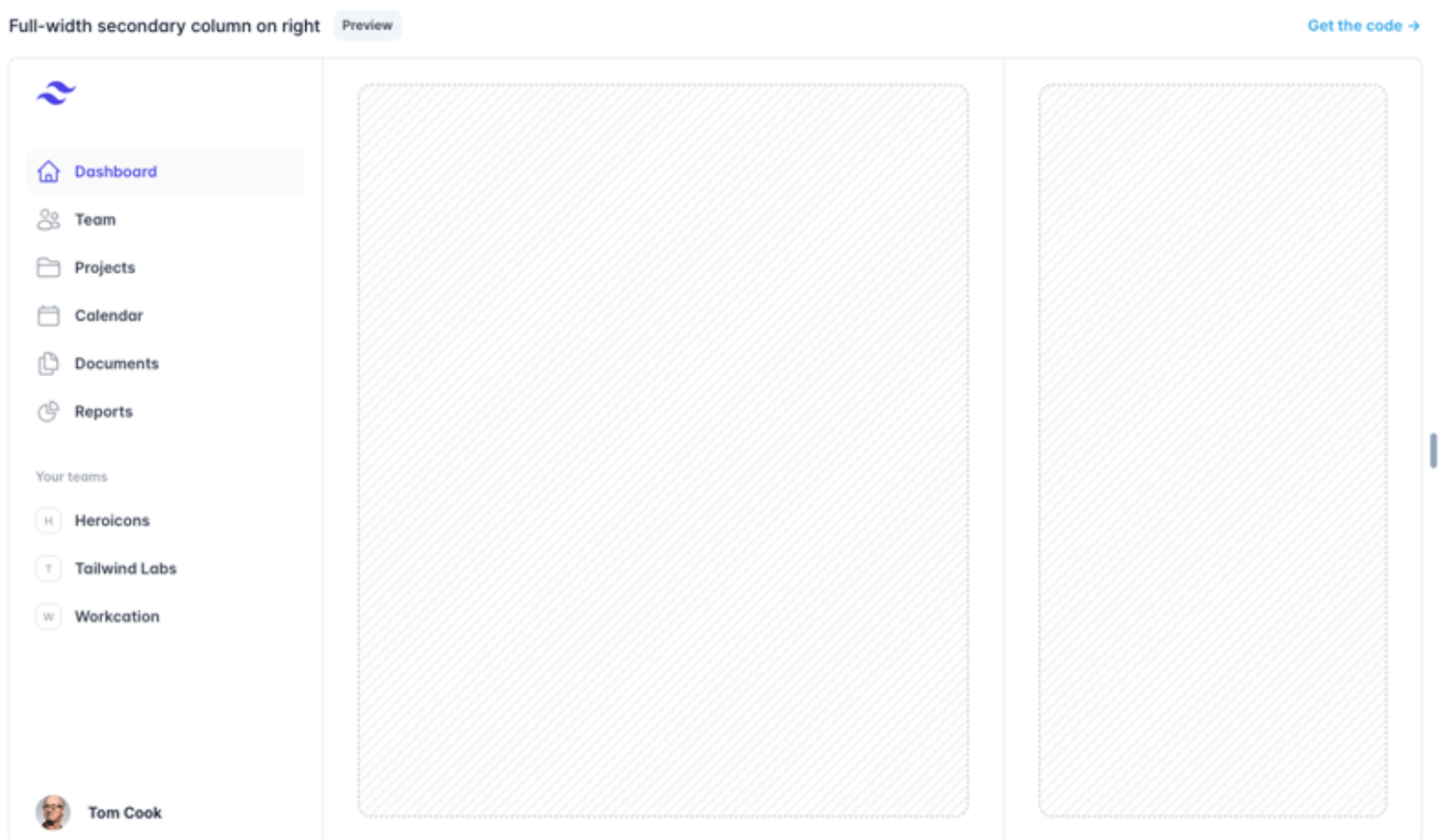
```
<label class="peer">
   <input type="checkbox" name="yo" checked="">
   Create a to do list
</label>
<svg class="peer-has-checked:hidden">...</svg>
```

Il est possible de nommer les pairs :

```
<label class="peer/first">
   <input type="checkbox" name="yo" checked="">
   Create a to do list
</label>
<svg class="peer-has-checked/first:hidden">...</svg>
```

Mise en pratique

Créer des composants et/ou templates inspirés de TailwindUI.



Pour les consignes et le chemin à suivre, rendez-vous sur myGES, dans la rubrique supports de cours : TP3-tailwind 🤟

Créer des classes utilitaires custom

Il est possible de définir ses propres classes utilitaires grâce à la directive **@utility**

Classe statique :

```
@utility esgi{
   background-color : var(--color-blue-500);
}
```

Depuis la v4, ces classes dynamiques sont aussi appelées classes **fonctionnelles**
Classe dynamique :

```
@utility esgi-*{
   background-color : --value(--color-*);
}
```

Dans cet exemple, la fonction **--value()** récupère le nom passé à la classe (2xl, blue-500...) et va chercher la valeur correspondante dans le namespace

Créer des classes utilitaires custom

La fonction **--value()**

```
@theme {
  --tab-size-2: 2;
  --tab-size-4: 4;
  --tab-size-github: 8;
}

@utility tab-* {
  tab-size: --value(--tab-size-*);
}
```

En plus des valeurs définies dans le thème, la fonction --value() peut accepter des valeurs brutes :

```
@utility tab-* {
  tab-size: --value(integer);
  tab-size: --value('inherit', 'initial', 'unset');
}
```

Créer des classes utilitaires custom

La fonction **--value()**

En plus des valeurs définies dans le thème, la fonction --value() peut accepter des valeurs brutes :

```
@utility tab-* {
  tab-size: --value(integer);
  tab-size: --value('inherit', 'initial', 'unset');
}
```

Ou des valeurs arbitraires :

```
@utility tab-* {
  tab-size: --value([integer]);
}
```

Créer des classes utilitaires custom

La fonction **--value()**

Une classes peut accepter tous les types de valeurs :

```
@utility opacity-* {
  opacity: --value([percentage]);
  opacity: calc(--value(integer) * 1%);
  opacity: --value(--opacity-*);
}
```

Les types de valeurs acceptées par tailwindcss :

- length
- integer
- number
- percentage
- ratio
- color
- angle
- url

Créer des classes utilitaires custom

Utiliser des **valeurs négatives**

```
class = "-m-4"
```

Pour utiliser des valeurs négatives, on fait figurer un "-" avant le nom de classe.
Dans le cas d'une classe custom, on fait comme cela :

```
@utility esgi-* {
  margin : calc( --value(integer) * var(--spacing) );
}

@utility -esgi-* {
  margin : calc( --value(integer) * var(--spacing) * -1);
}
```

Créer des classes utilitaires custom

La fonction **--modifier()**

On peut ajouter un modifier à une classe utilitaire :

```
class = "utility/modifier"
class = "bg-red-500/70"
```

Le *modifier* permet de changer le comportement de la classe grâce à la fonction --modifier()

```
@utility esgi-* {
  background-color: color-mix(in oklab, --value(--color-*) --modifier(integer)% , transparent);
}
```

Tout comme la fonction --value( ), --modifier( ) fait référence soit à une valeur du thème, soit à une valeur brute.

Bibliographie

Sass
https://sass-lang.com/

CSS
https://developer.mozilla.org/fr/docs/Web/CSS

BEM
https://getbem.com/introduction/

Tailwind css
https://tailwindcss.com/

Test live de Tailwind :
https://play.tailwindcss.com/